Statement:

We have a postgress database and I want to have its replica configured and backup routine do some R&D and let me know your findings so that we can start working on it and do remember you will be working on production so do your drill well so that you know the concerns.
PITR: Point In Time Recovery
WAL: Write Ahead Log

1. Versions

Supported PostgreSQL majors are 14 to 18. 14 expires November of this year, 15 in Nov, 2027.
The replica must run the same major version as the primary.

2. Replication Design

Add one replica server that receives every change from the main server within seconds. The main server will NOT wait for the replica, so any problem with the replica will never slow down or stop the application.
Plan:

1. Physical streaming replication to one hot standby on a separate server, ideally on separate hardware, power and storage.
2. Asynchronous to start. Typical lag is under a second on a healthy network.
3. A dedicated replication role with the REPLICATION attribute, SCRAM-SHA-256 password authentication, and a pg_hba.conf entry restricted to the replica IP address.

Failover options:

Options	Working	Pro	Con
Manual promotion	An engineer confirms the primary is down, fences it, promotes the standby, and repoints the application. Before promoting the standby, the old primary must be stopped or isolated so it can never accept writes again.	Simple, predictable, no extra components	Slower recovery; depends on someone being available
repmgr	Agent on each node monitors and can promote a standby	Lightweight, easier than Patroni	Weaker split-brain protection than consensus-based

			tools
Patroni (+ etcd)	Nodes elect a leader through a 3-node etcd cluster; automatic promotion and fencing	Industry standard for automatic HA; actively maintained	Needs at least 3 extra components to run and monitor; misconfiguration can cause unplanned failovers

3. Backup Design

The backup will be stored on a separate server, encrypted, because it contains customer information. Keep a simple weekly export that can be restored with standard PostgreSQL tools alone, as insurance in case a backup tool ever stops being supported.

Option	Working	Pro	Con
pg_dump / pg_restore	Portable export of one database	Restorable across major versions; simple; no extra tools	No PITR; slow to restore large databases; consistent only at dump start
pg_basebackup + WAL archive	Full physical copy plus continuous changes	Built into PostgreSQL; enables PITR	No retention management, compression policy or verification; scripts become hard to maintain
PostgreSQL 17 incremental backup	Changed blocks since the last backup	Native; smaller backups	Still needs WAL archiving for PITR; restore requires pg_combinebackup; no retention management
Barman (EDB)	Backup server with WAL streaming and PITR	Mature, multi-maintainer, good on-prem fit	Slower parallel and incremental restore than pgBackRest
pgBackRest	Full, differential and incremental backups, WAL archiving, PITR	Parallel backup and restore, delta restore, encryption, retention, verification	More setup, learning curve

4. Recommended

1. pgBackRest as the primary backup tool, on a dedicated backup server, with continuous WAL archiving, a weekly full and daily differential backup (to be tuned to database size), encryption at rest, and retention matching business requirements.
2. Barman documented and tested as the fallback, so a switch is possible without improvising during an incident.
3. Weekly pg_dump of each database, stored separately. This survives any backup tool becoming unavailable and supports restores into a different major version.
4. Scheduled restore tests to a separate server, with the restore time recorded, because an untested backup cannot be trusted.

5. Risks and Safeguards

Options	Working	Cause	Safeguard
WAL archiving fails silently	Saved changes pile up until the main server disk is full and the database stops	Failed archive_command causes WAL to be retained in pg_wal	Alert on pg_stat_archiver.failed_count and pg_wal size; test archiving before relying on it
Replica offline with a slot	The main server keeps changes for an absent replica until its disk fills	Inactive replication slot pins WAL	Set max_slot_wal_keep_size; alert on slot lag and inactive slots
Restart required	Turning on archiving briefly stops the database	archive_mode takes effect only at server start	Maintenance window, approval, and announcement
Load from the initial copy	Copying the whole database can slow the live system	pg_basebackup reads the full data directory	Run off-peak; throttle with --max-rate
Backups on the same server	If the server is lost, so are the backups	Single failure domain	Separate backup server plus encrypted off-site copy
Untested backups	Backups turn out to be unusable during a real incident	Corruption, missing WAL, wrong configuration	Scheduled restore drills with measured restore times
Replica queries slow	Reports on the replica can cause the main database	hot_standby_feedback delays vacuum on the primary	Decide deliberately; monitor bloat

cleanup	to grow		
Split-brain after failover	Two servers accept writes and data diverges	Old primary returns without being fenced	Fencing step in the runbook; rebuild old primary as a standby
Sensitive data in backups	Customer data is exposed if backup files leak	Unencrypted backups or broad access	Encryption at rest and in transit; restricted access to the backup server
Retention obligations	Backups deleted too early, or kept longer than allowed	Retention policy not aligned with regulation	Confirm retention with management before configuring
Version mismatch	Replica cannot be built or replication stops after an upgrade	Different major versions	Same major version on all nodes; plan upgrades as a project

6. Decision pending details

1. PostgreSQL version, operating system
2. Database size and daily write volume
3. Existing backups and whether a restore has ever been tested
4. Acceptable data loss (RPO) and downtime (RTO)
5. Servers available for the replica and backups
6. Retention requirements, including regulatory obligations
7. Maintenance window availability and change approver